



En esta parte de la materia nos dedicamos a *especificar* problemas. Para eso planteamos *procedimientos* que reciben datos de entrada y modifican algunos de ellos o devuelven datos de salida. Describiremos con lógica de primer orden las propiedades que tienen que cumplir los datos de entrada para que el programa se comporte adecuadamente (los *requiere*) y las propiedades que cumplirán los datos de salida (los *asegura*). Este documento contiene el detalle del lenguaje de especificación que vamos a usar. La mayor parte del documento está abocado a describir el lenguaje de primer orden que se utiliza para las propiedades de datos de entradas y salida.

Índice

1. Especificación de procedimientos	1
1.1. Tipos de parámetros	2
1.2. Predicados y funciones auxiliares	2
1.3. Cuantificadores y funciones especiales	3
2. Lenguaje de Primer Orden para cláusulas <i>requiere</i> y <i>asegura</i>	5
2.1. Tipos básicos	5
2.1.1. <code>bool</code> : valor booleano	6
2.1.2. $\mathbb{Z}$: número entero.	6
2.1.3. $\mathbb{R}$: número real.	6
2.1.4. <code>char</code> : caracter.	6
2.2. Tipos complejos	6
2.2.1. <code>struct</code> $\langle \text{campo}_1 : T_1, \dots, \text{campo}_n : T_n \rangle$	6
2.2.2. <code>tupla</code> $\langle T_1, \dots, T_n \rangle$	7
2.2.3. <code>seq</code> $\langle T \rangle$	7
2.2.4. <code>string</code> : renombre de <code>seq</code> $\langle \text{char} \rangle$	8
2.2.5. <code>conj</code> $\langle T \rangle$	8
2.2.6. <code>dicc</code> $\langle K, V \rangle$	9
2.3. Renombres de tipos y tipos enumerados	10
2.4. Sumatorias y productorias sobre conjuntos y diccionarios	10

1. Especificación de procedimientos

La definición de un procedimiento tiene tres partes:

- **Signatura.** Incluye el **nombre** del procedimiento, la lista de **parámetros** y el tipo de datos del **resultado**, si lo hubiera
- **Precondición.** Compuesta por las cláusulas *requiere* escritas en lógica de primer orden.
- **Postcondición.** Compuesta por las cláusulas *asegura* escritas en lógica de primer orden.

Veamos un ejemplo:

```
proc raizCuadrada(in x :  $\mathbb{R}$ ) :  $\mathbb{R}$  {
  requiere {  $x > 0$  }
  asegura {  $res \times res = x$  }
}
```

Esta especificación describe el comportamiento del procedimiento **raizCuadrada**, el cual recibe un dato de tipo real (x), que debe ser positivo ($x > 0$), y devuelve un valor de tipo real (res), que debe ser igual a la raíz cuadrada del valor de la entrada ($res \times res = x$).

Notar que la fórmula del *requiere* hace referencia a una variable x , a una constante 0 y un símbolo de predicado $>$. A su vez el *asegura* refiere a dos variables res y x , un símbolo de función $\times$ y un símbolo de predicado $=$. Los símbolos de función y predicado son parte del lenguaje de primer orden que definimos en la próxima sección.

1.1. Tipos de parámetros

Los parámetros de un procedimiento pueden ser de entrada (*in*) o entrada/salida (*inout*).

Si el procedimiento devuelve un valor se indica el tipo de dato del resultado por fuera de la lista de parámetros como resultado del procedimiento. Nos referiremos a este valor con la palabra reservada *res*. No tiene sentido referirse a su valor en los *requiere*.

```
proc doble(in a :  $\mathbb{Z}$ ) :  $\mathbb{Z}$  {
  asegura {  $res = 2 \times a$  }
}
```

Los parámetros de entrada (*in*) tienen un valor que puede leerse en cualquier momento y que no podrá ser modificado por el código del procedimiento, por lo que a la salida tendrá el mismo valor que a la entrada. Por lo tanto, es posible referirse al valor del mismo tanto en los *requiere* como en los *asegura*.

Los parámetros de entrada/salida (*inout*) pueden ser modificados por el procedimiento, pudiendo tener un valor de salida distinto al recibido en la entrada. Para referirnos al valor inicial utilizamos *metavariables*. Sea a una variable de tipo *inout*, definimos en el *requiere* el valor de una variable adicional (A_0) que preserva su valor y sobre la que podemos hacer referencia en los *asegura* para referirnos al valor inicial de a .

```
proc sumarUno(inout a :  $\mathbb{Z}$ ) {
  requiere {  $a = A_0$  }
  asegura {  $a = A_0 + 1$  }
}
```

En este ejemplo, la variable a tiene a la entrada un valor desconocido que denominaremos A_0 . A la salida el valor de a será igual al que tenía a la entrada más uno.

Es posible escribir muchas expresiones *requiere* y *asegura*. Las mismas se considerarán unidas con el conector $\wedge_L$ en el orden que fueron introducidas. A continuación se muestran dos especificaciones equivalentes.

```
proc recortarRango(inout s : seq< $\mathbb{Z}$ >, in desde :  $\mathbb{Z}$ , in hasta :  $\mathbb{Z}$ ) {
  requiere {  $0 \leq desde < |s|$  }
  requiere {  $0 \leq hasta < |s|$  }
  requiere {  $desde \leq hasta$  }
  asegura { ... }
}

proc recortarRango(inout s : seq< $\mathbb{Z}$ >, in desde :  $\mathbb{Z}$ , in hasta :  $\mathbb{Z}$ ) {
  requiere {  $0 \leq desde < |s| \wedge_L 0 \leq hasta < |s| \wedge_L desde \leq hasta$  }
  asegura { ... }
}
```

1.2. Predicados y funciones auxiliares

Para simplificar la escritura de predicados y facilitar su lectura y comprensión, es posible descomponerlos en funciones y predicados auxiliares.

Los predicados encapsulan fórmulas lógicas, y se evalúan a un valor de verdad. Por otro lado, las funciones auxiliares permiten encapsular una operatoria que tiene como resultado un elemento de un tipo específico. Veamos un ejemplo:

```

proc distanciaEntrePuntos2D(in x1 : ℤ, in y1 : ℤ, in x2 : ℤ, in y2 : ℤ) : ℤ {
  requiere {
    EsPositivo(x1) ∧ EsPositivo(y1) ∧ EsPositivo(x2) ∧ EsPositivo(y2)
  }
  asegura { res = Dist(x1, y1, x2, y2) }
}

pred EsPositivo(x : ℤ) { x > 0 }

aux Dist(x1 : ℤ, y1 : ℤ, x2 : ℤ, y2 : ℤ) : ℤ =  $\sqrt{(x2 - x1)^2 + (y2 - y1)^2}$ 

```

Importante: a diferencia de los procedimientos, los predicados y funciones auxiliares *no describen problemas*. Son simples herramientas sintácticas para descomponer predicados. Nunca modifican los parámetros, por lo que no es necesario indicar el tipo de pasaje (*in* o *inout*) como lo hacíamos con los *procs*.

El predicado anterior es *equivalente* a reemplazar el cuerpo en el predicado que lo referencia:

```

proc distanciaEntrePuntos2D(in x1 : ℤ, in y1 : ℤ, in x2 : ℤ, in y2 : ℤ) : ℤ {
  requiere {
    x1 > 0 ∧ y1 > 0 ∧ x2 > 0 ∧ y2 > 0
  }
  asegura {
    res =  $\sqrt{(x2 - x1)^2 + (y2 - y1)^2}$ 
  }
}

```

Formalmente, los predicados y funciones auxiliares son *azúcar sintáctica*, no agregan expresividad al lenguaje de primer orden sobre el que estamos construyendo fórmulas. El lenguaje de primer orden base que definiremos en la próxima sección está fijo y las definiciones de auxiliares las podemos reemplazar (como en el ejemplo de arriba), al momento de evaluar la validez de una fórmula.

Importantísimo: no se puede utilizar una referencia a un procedimiento desde los requiere o asegura de otro procedimiento. Tampoco desde un predicado auxiliar. Formalmente, no tiene sentido: una cláusula *requiere* o *asegura* es una fórmula sobre un lenguaje de primer orden. El procedimiento no es parte de ese lenguaje. Tampoco tiene sentido pensar en hacer un reemplazo sintáctico del nombre de un procedimiento por su definición dentro de una fórmula: ¿cómo “combinamos” su pre y postcondición?

El siguiente ejemplo es incorrecto.

```

proc máximo(in s : seq(ℤ)) : ℤ {
  requiere { ... }
  asegura { ... }
}

proc posiciónMáximo(in s : seq(ℤ)) : ℤ {
  requiere { |s| > 0 }
  asegura { s[res] = máximo(s) // INCORRECTO }
}

```

1.3. Cuantificadores y funciones especiales

Para escribir los predicados de los requiere y asegura usaremos lógica trivaluada de primer orden. Los **cuantificadores** que usaremos son los siguientes:

Operación	Sintaxis	Significado
cuantificador universal	$(\forall i : T) (P(i))$	Todo valor i de tipo T tiene que cumplir el predicado $P(i)$
cuantificador existencial	$(\exists i : T) (P(i))$	Existe al menos un valor i de tipo T que cumple el predicado $P(i)$

Algunos ejemplos:

- $(\forall n : \mathbb{Z}) (n \times n \geq n)$
Todo número entero cumple que su cuadrado es mayor o igual a sí mismo.
- $(\forall n : \mathbb{Z}) (n \bmod 4 = 0 \rightarrow n \bmod 2 = 0)$
Todo número entero cumple que si es divisible por 4, entonces es divisible por 2.
- $(\exists i : \mathbb{Z}) (10 \bmod i = 0)$
Existe un número entero que cumple que 10 es divisible por él.

Es posible cuantificar sobre múltiples variables y anidar cuantificadores.

- $(\forall n, m : \mathbb{Z}) ((n > 0 \wedge m > 0 \wedge n) < m \rightarrow n^2 < m^2)$
Para todos dos números positivos n y m que cumplen con que n es menor a m , entonces el cuadrado de n es menor que el cuadrado de m .
- $(\forall n : \mathbb{Z}) (esPar(n) \rightarrow (\exists m : \mathbb{Z}) (2 \times m = n))$
Para todo número entero par n , siempre existe un número m multiplicado por 2 es igual a n .
- $(\exists n : \mathbb{Z}) (esPar(n) \wedge (\forall m : \mathbb{Z}) (esPar(m) \rightarrow m \bmod n = 0))$
Existe un número par que divide a todos los números pares

Muy frecuentemente vamos a usar cuantificadores para describir el contenido de **secuencias**. Por ejemplo:

- $(\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L s[i] > 0)$
Los elementos en todas las posiciones de la secuencia s son mayores que cero. Usamos el operador $\rightarrow_L$ porque en caso de que se haga falso el antecedente (i no está en el rango) el consecuente podría indefinirse, ya que se intenta indexar en una posición inválida de la secuencia s .
- $(\forall n : \mathbb{Z}) (n \in s \rightarrow n > 0)$
Este predicado es equivalente al anterior pero en lugar de utilizar índices para recorrer la secuencia usamos *pertenece* ($\in$). Notar que ya no es necesario usar el $\rightarrow_L$ porque ahora no hay nada que se pueda indefinir en el consecuente.
- $(\exists i : \mathbb{Z}) (0 \leq i < |s| \wedge i \bmod 2 = 0 \wedge_L s[i] \bmod 3 = 0)$
Existe una posición par de la secuencia s que contiene un elemento divisible por 3.
- $(\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L (\exists k : \mathbb{Z}) (k \times k = s[i]))$
Todos los elementos de la secuencia s son cuadrados perfectos.

Una **función especial** que usaremos es la auxiliar **IfThenElse**. Esta función evalúa una condición y devuelve el primer valor si la condición es verdadera y el segundo si la condición es falsa. La condición puede ser cualquier predicado y los dos valores deben ser del mismo tipo. Nótese que el tipo de la expresión completa será el mismo que el de los valores.

Algunos ejemplos:

- **IfThenElse**($x > 0, x, -x$)
Devuelve el valor absoluto de x .
- **IfThenElse**($x_1 > x_2, x_1 - x_2, x_2 - x_1$) + **IfThenElse**($y_1 > y_2, y_1 - y_2, y_2 - y_1$)
Devuelve la distancia de Manhattan (sobre una grilla) entre los puntos (x_1, y_1) y (x_2, y_2) .

Notas:

- Antes de usar un **IfThenElse**() intentar resolverlo mediante operadores lógicos. Por ejemplo $res = \text{IfThenElse}(p, a, b)$ es equivalente a $(p \wedge res = a) \vee (\neg p \wedge res = b)$
- Esta función *no se utiliza* para describir causalidad (si pasa P entonces se cumple Q), ya que la evaluación de la función *if* devuelve un valor de algún tipo. La forma correcta de expresar causalidad es utilizando la implicación, de la forma $P \rightarrow Q$.

- Tampoco se debe usar para “ejecutar” operaciones diferentes según una condición, es decir, lo correcto es $s[i] = \text{IfThenElse}(\text{esPar}(i), S_0[i]/2, S_0[i] + 3)$ en lugar de $\text{IfThenElse}(\text{esPar}(i), s[i] = S_0[i]/2, s[i] = S_0[i] + 3)$

Por último, para operar con los elementos de secuencias, vamos a usar las siguientes **funciones especiales**:

Operación	Sintaxis	Significado
sumatoria	$\sum_{i=j}^n f(i)$	Equivalente a $f(j) + f(j+1) + \dots + f(n)$
productoria	$\prod_{i=j}^n f(i)$	Equivalente a $f(j).f(j+1).\dots.f(n)$

Ejemplos:

- $\sum_{i=0}^{100} i$
La suma de todos los enteros entre 0 y 100.
- $\prod_{i=1}^{10} 2 \times i$
El producto de los primeros 10 números pares.
- $\sum_{i=0}^{|s|-1} s[i]$
La suma de todos los elementos de la secuencia s .

Si se combinan estas funciones especiales con el `IfThenElse` se pueden agregar condiciones o incluso contar los elementos que cumplan una determinada condición:

- $\sum_{i=0}^{|s|-1} \text{IfThenElse}(s[i] \bmod 2 = 0, 1, 0)$
La cantidad de elementos pares en la secuencia s .

Aclaración. Para cualquiera de los dos casos, siendo a y b los límites inferior y superior respectivamente de las funciones especiales, si ocurre que $a > b$ (rango vacío) la sumatoria y productoria se reducen al neutro de la función que corresponde, independientemente de la $f(i)$ que se encuentre dentro.

$$\sum_{i=a}^b (\dots) = 0$$

$$\prod_{i=a}^b (\dots) = 1$$

2. Lenguaje de Primer Orden para clausulas *requiere* y *asegura*

En esta sección presentamos el lenguaje de primer orden que define las fórmulas que pueden utilizarse en un *requiere* o *asegura*. Es decir, definimos todos los tipos, las constantes y los símbolos de función y de predicado que son **síntacticamente válidos** y por lo tanto pueden utilizarse en una fórmula.

No proveemos una sección separada para definir una **interpretación de los símbolos** a ‘el mundo real’. En vez, definimos la interpretación implícitamente, utilizando nombres de tipos, constantes, funciones y predicados que remitan a teorías matemáticas conocidas. A veces usamos lenguaje natural para dar información complementaria que ayuda a interpretar correctamente los símbolos.

Organizamos los símbolos por tipo al que se aplican.

2.1. Tipos básicos

Las constantes devuelven un valor del tipo. Las funciones operan con elementos de los tipos y retornan algun elemento de algun tipo. Los predicados generan fórmulas a partir de elementos de tipo.

2.1.1. **bool**: valor booleano

Símbolo	Sintaxis
constantes	$True, False$
funciones	$\wedge, \vee, \neg, \rightarrow, \leftrightarrow$
predicados	$=, \neq$

2.1.2. $\mathbb{Z}$: número entero.

Símbolo	Sintaxis
constantes	$1, 2, \dots$
funciones	$+, -, \cdot, /$ (div. entera), mód (módulo), $\dots$
predicados	$<, >, \leq, \geq, =, \neq$

2.1.3. $\mathbb{R}$: número real.

Símbolo	Sintaxis
constantes	$1, 2, \dots$
funciones	$+, -, \cdot, /, \sqrt{x}, \sin(x), \dots$
predicados	$<, >, \leq, \geq, =, \neq$

2.1.4. **char**: caracter.

Símbolo	Sintaxis
constantes	$'a', 'b', 'A', 'B'$
funciones	$\text{ord}(c), \text{char}(c)$
predicados (a partir de ord)	$<, >, \leq, \geq, =, \neq$

2.2. Tipos complejos

2.2.1. **struct** $\langle \text{campo}_1 : T_1, \dots, \text{campo}_n : T_n \rangle$

Tupla con nombres para los campos.

Símbolo	Sintaxis
struct por extensión	$\langle x : 20, y : 10 \rangle$
obtener el valor de un campo	$s.x, s.y$
igualdad	$s_1 = s_2$

- struct por extensión: permite crear structs a partir de sus campos
 - sintaxis: $\langle \text{campo}_1 : T_1, \text{campo}_2 : T_2, \dots \rangle : \text{struct } \langle \text{campo}_1 : T_1, \text{campo}_2 : T_2, \dots \rangle$
 - ejemplos: $\langle x : 20, y : 10 \rangle$ (struct con dos campos de tipo $\mathbb{Z}$ denominados x e y con los valores 20 y 10 respectivamente)
- obtener el valor de un campo: devuelve el valor de un campo de la struct. Se indefinido si el campo no existe.
 - sintaxis: $\langle \text{campo}_1 : T_1, \dots, \text{campo}_n : T_n \rangle.\text{campo}_i : T_i$
 - ejemplos: $\langle x : 20, y : 10 \rangle.x = 20$, $\langle x : 20, y : 10 \rangle.z$ es indefinido
- igualdad: Resulta *Verdadero* sii s_1 y s_2 tienen
 - misma cantidad de elementos
 - con los mismos nombres
 - con el mismo tipo para cada nombre
 - con los mismos valores para cada nombre

Nota: El orden de los campos del struct no es relevante para la igualdad, pero les recomendamos¹ que siempre escriban los campos en el mismo orden.

¹Les pedimos encarecidamente

2.2.2. tupla $\langle T_1, \dots, T_n \rangle$

Tupla de tipos $T_1, \dots, T_n$

Símbolo	Sintaxis
tupla por extensión	$\langle x, y, z \rangle$
obtener un valor	s_i
igualdad	$t_1 = t_2$

- tupla por extensión: permite crear tuplas a partir de sus elementos
 - sintaxis: $\langle x : T_1, y : T_2, \dots \rangle : \text{tupla } \langle T_1, T_2, \dots \rangle$
 - ejemplos: $\langle 1, 'hola', 25, 45 \rangle$ (tupla de tipo $\langle \mathbb{Z}, \text{string}, \mathbb{R} \rangle$ con los valores 1, 'hola' y 25.45)
- obtener un valor: devuelve el valor en una determinada posición de la tupla. El primer elemento es el 0. Se indefinire si el índice es menor a cero o mayor a la cantidad de elementos de la tupla.
 - sintaxis: $\langle T_1, \dots, T_n \rangle_{i:\mathbb{Z}} : T_i$
 - ejemplos: $\langle 1, 'hola', 25, 45 \rangle_0 = 1$, $\langle 1, 'hola', 25, 45 \rangle_2 = 25, 45$, $\langle 1, 'hola', 25, 45 \rangle_5$ es indefinido
- igualdad: Resulta *Verdadero* si t_1 y t_2 tienen
 - misma cantidad de elementos
 - todos del mismo tipo
 - con los mismos valores

NOTA: La tupla tiene la misma funcionalidad del struct, pero es *menos declarativa*. Recomendamos usarla sólo en casos muy triviales donde es muy evidente el significado de cada posición de la tupla.

2.2.3. $\text{seq}\langle T \rangle$

Secuencia de tipo T

Símbolo	Sintaxis
secuencia por extensión	$\langle \rangle, \langle x, y, z \rangle$
longitud	$ s , \text{length}(s)$
pertenece	$i \in s$
indexación	$s[i]$
igualdad	$s_1 = s_2$
cabeza	$\text{head}(s)$
cola	$\text{tail}(s)$
concatenación	$\text{concat}(s_1, s_2), s_1 ++ s_2$
subsecuencia	$\text{subseq}(s, i, j)$
cambiar elemento	$\text{setAt}(s, i, \text{val})$

- secuencias por extensión: permite crear secuencias a partir de sus elementos
 - sintaxis: $\langle x : T, y : T, z : T, \dots \rangle : \text{seq}\langle T \rangle$
 - ejemplos: $\langle \rangle$ (secuencia vacía). $\langle 1, 2, 25 \rangle$ (secuencia de enteros con tres elementos: 1, 2 y 25)
- longitud: devuelve la cantidad de elementos de la secuencia
 - sintaxis: $|s : \text{seq}\langle T \rangle| : \mathbb{Z}$. Alternativa: $\text{length}(s)$ o $\text{longitud}(s)$
 - ejemplos: $|\langle \rangle| = 0$, $|\langle 1, 2, 25 \rangle| = 3$
- pertenece: indica si un elemento está en la secuencia
 - sintaxis: $i : T \in s : \text{seq}\langle T \rangle$ (devuelve un valor de verdad)
 - ejemplos: $1 \in \langle \rangle$ es falso, $1 \in \langle 2, 3 \rangle$ es falso, $1 \in \langle 1, 2, 3 \rangle$ es verdadero
- indexación: devuelve el elemento en una determinada posición de la secuencia. El primer elemento es el 0. Se indefinire si el índice es menor a cero o mayor al tamaño de la secuencia.
 - sintaxis: $\text{seq}\langle T \rangle[i : \mathbb{Z}] : T$.
 - ejemplos: $\langle 1, 2, 3 \rangle[0] = 1$, $\langle 1, 2, 3 \rangle[2] = 3$, $\langle 1, 2, 3 \rangle[5]$ es indefinido

- igualdad: Resulta *Verdadero* sii s_1 y s_2 :
 - son del mismo tipo T
 - son del mismo largo
 - para cada posición i , $s_1[i] = s_2[i]$
- cabeza: devuelve el primer elemento de la secuencia. Se indefinire si la secuencia es vacía
 - sintaxis: $head(s : seq\langle T \rangle) : T$
 - ejemplos: $head(\langle 1, 2, 3 \rangle) = 1$, $head(\langle \rangle)$ es indefinido
- cola: devuelve la secuencia sin el primer elemento. Se indefinire si la secuencia es vacía
 - sintaxis: $tail(s : seq\langle T \rangle) : seq\langle T \rangle$
 - ejemplos: $tail(\langle 1, 2, 3 \rangle) = \langle 2, 3 \rangle$, $tail(\langle 1 \rangle) = \langle \rangle$, $tail(\langle \rangle)$ es indefinido
- concatenación: concatena dos secuencias
 - sintaxis: $concat(s_1 : seq\langle T \rangle, s_2 : seq\langle T \rangle) : seq\langle T \rangle$. Alternativa: $s_1 ++ s_2$
 - ejemplos: $concat(\langle 1, 2 \rangle, \langle 3, 4 \rangle) = \langle 1, 2, 3, 4 \rangle$, $\langle \rangle ++ \langle 1, 2 \rangle = \langle 1, 2 \rangle$
- subsecuencia: devuelve una subsecuencia de la secuencia original, incluyendo el índice del principio y sin incluir el índice del final. Se indefinire si los índices están fuera de rango o si el índice del final es menor al del principio. Devuelve una secuencia vacía si los índices son iguales.
 - sintaxis: $subseq(s : seq\langle T \rangle, i : \mathbb{Z}, j : \mathbb{Z}) : seq\langle T \rangle$
 - ejemplos: $subseq(\langle 1, 2, 3, 4, 5 \rangle, 1, 3) = \langle 2, 3 \rangle$, $subseq(\langle 1, 2, 3, 4, 5 \rangle, 1, 1) = \langle \rangle$, $subseq(\langle 1, 2, 3, 4, 5 \rangle, 3, 1)$ es indefinido
- cambiar elemento: devuelve una secuencia igual a la original pero con un elemento cambiado. Se indefinire si el índice está fuera de rango.
 - sintaxis: $setAt(s : seq\langle T \rangle, i : \mathbb{Z}, val : T) : seq\langle T \rangle$
 - ejemplos: $setAt(\langle 1, 2, 3 \rangle, 1, 5) = \langle 1, 5, 3 \rangle$, $setAt(\langle 1, 2, 3 \rangle, 5, 5)$ es indefinido

2.2.4. **string:** renombre de $seq\langle char \rangle$.

2.2.5. $conj\langle T \rangle$

Conjunto de tipo T

Símbolo	Sintaxis
conjunto por extensión	$\langle \rangle, \langle x, y, z \rangle$
tamaño	$ c $
pertenece	$i \in c$
igualdad	$c_1 = c_2$
inclusión	$c_1 \subseteq c_2$
union	$c_1 \cup c_2$
intersección	$c_1 \cap c_2$
diferencia	$c_1 - c_2$

- conjunto por extensión: permite crear conjuntos a partir de sus elementos
 - sintaxis: $\langle x : T, y : T, z : T, \dots \rangle : conj\langle T \rangle$
 - ejemplos: $\langle \rangle$ (conjunto vacío). $\langle 1, 2, 25 \rangle$ (conjunto de enteros con tres elementos: 1, 2 y 25)
- tamaño: devuelve la cantidad de elementos del conjunto
 - sintaxis: $|c : conj\langle T \rangle| : \mathbb{Z}$. Alternativa: $size(s)$ o $tamaño(s)$
 - ejemplos: $|\langle \rangle| = 0$, $|\langle 1, 2, 25 \rangle| = 3$
- pertenece: indica si un elemento está en el conjunto
 - sintaxis: $i : T \in c : conj\langle T \rangle$ (devuelve un valor de verdad)
 - ejemplos: $1 \in \langle \rangle$ es falso, $1 \in \langle 2, 3 \rangle$ es falso, $1 \in \langle 1, 2, 3 \rangle$ es verdadero
- igualdad: resulta *Verdadero* sii c_1 y c_2

- son del mismo tipo T
 - tienen la misma cantidad de elementos
 - $t \in c_1 \leftrightarrow t \in s_2[i]$
- inclusión: resulta *Verdadero* sii dados c_1 y c_2
- son del mismo tipo T
 - todos los elementos de c_1 están en c_2
- unión de conjuntos
- sintaxis: $c1 : \text{conj}\langle T \rangle \cup c2 : \text{conj}\langle T \rangle : \text{conj}\langle T \rangle$
 - ejemplos: $\langle 1, 2, 3 \rangle \cup \langle 1, 4, 5 \rangle = \langle 1, 2, 3, 4, 5 \rangle$, $\langle 1, 2, 3 \rangle \cup \langle \rangle = \langle 1, 2, 3 \rangle$
- intersección de conjuntos
- sintaxis: $c1 : \text{conj}\langle T \rangle \cap c2 : \text{conj}\langle T \rangle : \text{conj}\langle T \rangle$
 - ejemplos: $\langle 1, 2, 3 \rangle \cap \langle 1, 4, 5 \rangle = \langle 1 \rangle$, $\langle 1, 2, 3 \rangle \cap \langle \rangle = \langle \rangle$
- diferencia de conjuntos
- sintaxis: $c1 : \text{conj}\langle T \rangle - c2 : \text{conj}\langle T \rangle : \text{conj}\langle T \rangle$
 - ejemplos: $\langle 1, 2, 3 \rangle - \langle 1, 4, 5 \rangle = \langle 2, 3 \rangle$, $\langle 1, 2, 3 \rangle - \langle \rangle = \langle 1, 2, 3 \rangle$

2.2.6. $\text{dicc}\langle K, V \rangle$

Diccionario que asocia claves de tipo K con valores de tipo V .

Símbolo	Sintaxis
diccionario por extensión	$\langle \rangle, \langle \text{"juan"} : 20, \text{"diego"} : 10 \rangle$
tamaño	$ d $
pertenece (hay clave)	$k \in d$
igualdad	$d_1 = d_2$
valor	$d[k]$
claves	$\text{claves}(d)$
setear valor	$\text{setKey}(d, k, v)$
eliminar valor	$\text{delKey}(d, k)$

- diccionario por extensión: permite crear diccionarios a partir de sus elementos
- sintaxis: $\{k_1 : v_1, k_2 : v_2, \dots\} : \text{dicc}\langle K, V \rangle$
 - ejemplos: $\{\text{"juan"} : 20, \text{"diego"} : 10\}$ (diccionario de tipo $K=\text{string}$, $V=\mathbb{Z}$)
- tamaño: devuelve la cantidad de elementos (claves) del diccionario
- sintaxis: $|c : \text{dicc}\langle K, V \rangle| : \mathbb{Z}$. Alternativa: $\text{size}(s)$ o $\text{tamaño}(s)$
 - ejemplos: $|\{\}| = 0$, $|\{\text{"juan"} : 20, \text{"diego"} : 10\}| = 2$
- pertenece: indica si una clave está en el diccionario
- sintaxis: $k : K \in d : \text{dicc}\langle K, V \rangle$ (devuelve un valor de verdad)
 - ejemplos: $\text{"juan"} \in \{\}$ es falso, $\text{"juan"} \in \{\text{"juan"} : 20, \text{"diego"} : 10\}$ es verdadero
- igualdad: resulta *Verdadero* sii d_1 y d_2
- son de los mismos tipos K y V
 - tienen las mismas claves, es decir $\text{claves}(d_1) = \text{claves}(d_2)$
 - todas las claves tienen los mismos valores, es decir para todo k , $d_1[k] = d_2[k]$
- valor: devuelve el valor asociado con una clave. Se indefin si la clave no está en el diccionario
- sintaxis: $d[k : K] : V$
 - ejemplos: si $d = \langle \text{"juan"} : 20, \text{"diego"} : 10 \rangle$: $d[\text{"juan"}] = 20$, $d[\text{"ana"}]$ está indefinido
- claves: devuelve el conjunto de claves de un diccionario

- sintaxis: $claves(d : \text{dicc}\langle K, V \rangle) : \mathbb{Z}$
 - ejemplos: si $d = \langle \text{"juan"} : 20, \text{"diego"} : 10 \rangle$: $claves(d) = \langle \text{"juan"}, \text{"diego"} \rangle$
- **setKey**: Al igual que *setAt*, la función $setKey(d, k, v)$ devuelve un diccionario igual al ingresado pero con el valor de la clave k seteado en v . Es decir, para toda clave k' tal que $k' \in d \vee k' = k$:

$$setKey(d, k, v)[k'] = \begin{cases} v & \text{si } k' = k \\ d[k'] & \text{si } k' \neq k \end{cases}$$

- sintaxis: $setKey(d : \text{dicc}\langle K, V \rangle, k : K, v : V) : \text{dicc}\langle K, V \rangle$
 - ejemplos: si $d = \langle \text{"juan"} : 20, \text{"diego"} : 10 \rangle$: $setKey(d, \text{"juan"}, 5) = \langle \text{"juan"} : 5, \text{"diego"} : 10 \rangle$
- **delKey**: La función $delKey(d, k)$ elimina una clave del diccionario y deja igual todo el resto de los valores.
- sintaxis: $delKey[d : \text{dicc}\langle K, V \rangle, k : K] : \text{dicc}\langle K, V \rangle$
 - ejemplos: si $d = \langle \text{"juan"} : 20, \text{"diego"} : 10 \rangle$: $delKey(d, \text{"juan"}) = \langle \text{"diego"} : 10 \rangle$

2.3. Renombres de tipos y tipos enumerados

Para facilitar la lectura de las especificaciones se recomienda usar *renombres de tipos*. La sintaxis es:

Ranking **ES** $\mathbb{Z}$
 Posición **ES struct** $\langle x : \mathbb{R}, y : \mathbb{R} \rangle$

Para casos donde queremos representar un conjunto acotado de valores se pueden usar *tipos enumerados*, que sólo pueden tomar los valores definidos, y se pueden comparar entre sí únicamente por igualdad.

enum PuntoCardinal **ES** {Sur, Norte, Este, Oeste}

2.4. Sumatorias y productorias sobre conjuntos y diccionarios

Se puede escribir sumatorias y productorias sobre conjuntos usando la siguiente sintaxis

$$elementosPositivos = \sum_{e \in c} (\text{IfThenElse}(e \geq 0, 1, 0))$$

Para hacer algo similar sobre un diccionario hay que usar el diccionario de claves

$$significadosPositivos = \sum_{c \in claves(d)} (\text{IfThenElse}(d[c] \geq 0, 1, 0))$$